

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

«Системы искусственного интеллекта»

Отчет по лабораторной работе №4

Выполнила:
студентка
гр. ИСТбд-31
Феофанова П.А.

Проверил:
Шишкин В.В.

Ульяновск, 2026 г.

Введение

В данной лабораторной работе исследуются инструменты машинного обучения библиотеки Scikit-learn для решения задач классификации и кластеризации. Рассматриваются два классификатора по варианту — k -ближайших соседей (k -NN) и метод опорных векторов (SVC). Проводится сравнение качества моделей на исходных (“сырых”) данных и после предобработки, а также анализ результатов в табличной и графической форме. Дополнительно на тех же данных решается задача кластеризации средствами Scikit-learn.

Цель работы

- Изучить и применить инструменты классификации и кластеризации библиотеки Scikit-learn.
- Обучить и сравнить два классификатора (k -NN и SVC) на нескольких датасетах.
- Оценить влияние предобработки данных и разных способов разбиения на train/test на качество моделей.
- Выполнить кластеризацию и проанализировать результаты.

Постановка задачи

1. Выбрать и применить два классификатора: k -NN и SVC.
2. Использовать 3 датасета для экспериментов.
3. Для каждого датасета определить:
 - целевой столбец Y (если он присутствует),
 - набор признаков $X_1..X_7$,
 - при необходимости преобразовать типы данных.
4. Подготовить данные к обучению и провести обучение/оценку моделей:
 - на сырых данных (без масштабирования),
 - после предобработки (масштабирование, обработка пропусков),
 - с использованием разных методов формирования train/test (holdout и кросс-валидация).
5. Представить результаты в табличной и графической форме.
6. На тех же данных решить задачу кластеризации (например, KMeans), проанализировать результаты и сформулировать выводы.

Алгоритмическая часть

1) Классификация (k-NN и SVC)

Вход: датасет с признаками $X_1..X_7$ и целевым столбцом Y (0/1).

Шаги:

1. Загрузка данных из CSV/TSV.
2. Выделение признаков $X = (X_1..X_7)$ и целевой переменной $y = Y$.
3. Проверка пропусков (`isnull().sum()`).
4. Оценка на сырых данных (holdout):
 - `train_test_split` на обучающую и тестовую выборки,
 - обучение k-NN, обучение SVC,
 - предсказание на тесте,
 - расчёт accuracy и построение матрицы ошибок,
 - измерение времени обучения+предсказания.
5. Предобработка и оценка (CV + Scaling):
 - создание Pipeline(StandardScaler + модель),
 - StratifiedKFold для сохранения долей классов,
 - `cross_val_score` (средняя точность по фолдам),
 - измерение времени кросс-валидации.
6. Сравнение результатов в таблицах и на графиках (качество и время).

Почему масштабирование важно:

- Для k-NN расстояния между объектами критичны, разные масштабы признаков искажают вклад.
- Для SVC (особенно с RBF-ядром) масштабы признаков влияют на форму разделяющей поверхности и качество.

Структура данных и входной формат

1) Структура данных

- Данные хранятся в `pandas.DataFrame`.
- X — таблица признаков размерности $n \times 7$.
- y — вектор меток класса размерности n .

2) Входной формат

- Файл для датасетов 1 и 3: столбцы $X_1 X_2 X_3 X_4 X_5 X_6 X_7 Y$.

- Файл для датасета 2: столбцы X1..X7 (без Y, используется для кластеризации).
- Числа могут быть записаны с десятичной запятой, поэтому при чтении указывается `decimal=","`.
- Разделитель: табуляция;/, — выбирается в зависимости от сохранения файла (`sep="\t"` или `sep=";"`).

Пример строки:

1,0 2,0 27,1 102,0 2,0 130,0 12,08 0

Описание программы

Программа реализована на Python с использованием библиотек:

- `pandas` — загрузка и обработка данных,
- `scikit-learn` — модели классификации (k-NN, SVC), кросс-валидация, метрики,
- `matplotlib` — построение графиков.

Функциональные блоки:

- загрузка датасета и формирование X, y,
- обучение и оценка на сырых данных (holdout),
- предобработка данных через `Pipeline(StandardScaler, model)`,
- вывод таблиц (ассигасу, время) и построение графиков сравнения,
- кластеризация, расчёт метрик.

Экспериментальная часть

Исходные данные и подготовка

Для эксперимента использован датасет со столбцами признаков X1–X7 и целевым столбцом Y (бинарная классификация).

Проверка пропусков показала, что пропущенные значения отсутствуют во всех столбцах (X1...X7, Y = 0).

Данные были оценены двумя способами:

- Raw (Holdout): разбиение `train_test_split (test_size = 0.2, random_state = 42)`.
- CV + Scaling: кросс-валидация `StratifiedKFold(n_splits=5)` с предобработкой `StandardScaler` в `Pipeline`.

Результаты на сырых данных (Raw / Holdout)

k-NN (n_neighbors = 5)

- Accuracy: 0.8119
- Время: 0.0130 сек
- Матрица ошибок:

Pred 0	Pred 1	
True 0	246	22
True 1	38	13

Интерпретация: модель в целом работает удовлетворительно, но заметна доля ошибок по классу 1 (FN = 38), то есть часть объектов класса 1 классифицируется как 0.

SVC (RBF, C=1.0, gamma="scale")

Accuracy: 0.8401

Время: 0.0842 сек

Матрица ошибок:

Pred 0	Pred 1	
True 0	268	0
True 1	51	0

Интерпретация: классификатор предсказал только класс 0 (столбец Pred 1 полностью нулевой). Высокая accuracy в этом случае может быть обусловлена дисбалансом классов (если в данных много нулей). При этом качество распознавания класса 1 фактически отсутствует (TP = 0).

Результаты после предобработки (CV + Scaling)

Предобработка: стандартизация признаков StandardScaler. Оценка качества выполнена по 5-fold Stratified CV (усреднение accuracy по фолдам).

k-NN + Scaling

- Средняя Accuracy (CV): 0.8320

- Время: 0.1259 сек

Сравнение с holdout: точность выросла с 0.8119 до 0.8320. Это ожидаемо, так как k-NN чувствителен к масштабу признаков: стандартизация делает вклад признаков в расстояние более корректным.

SVC + Scaling

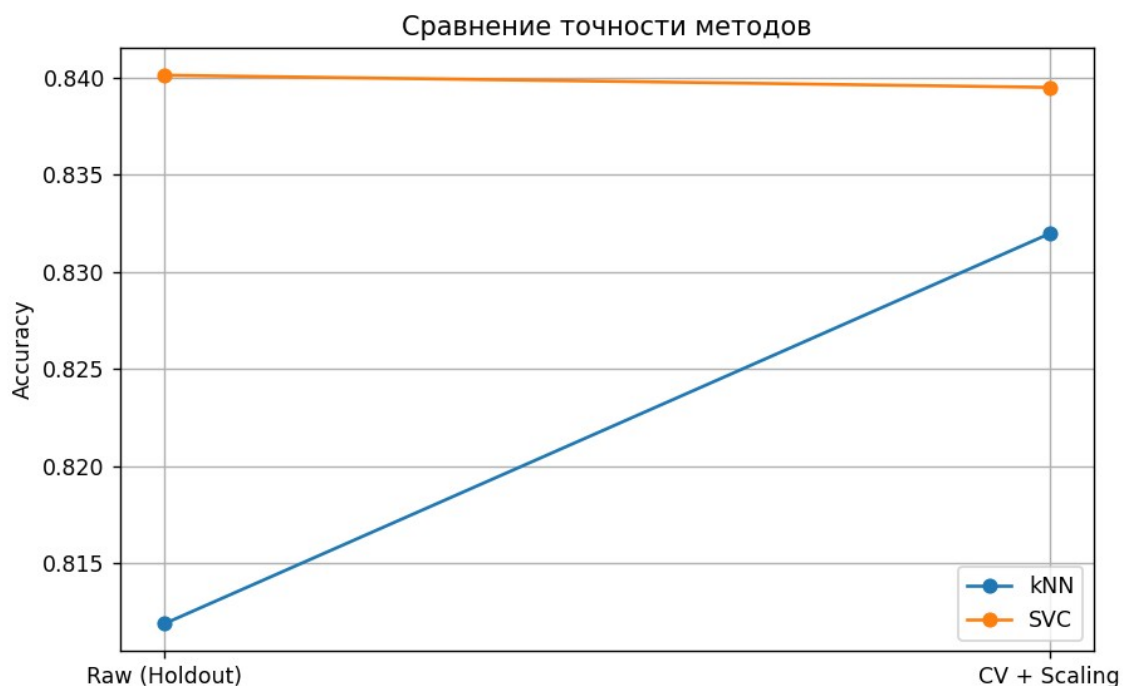
- Средняя Ассурасу (CV): 0.8395
- Время: 0.4728 сек

Сравнение с holdout: точность практически не изменилась (0.8401 → 0.8395), однако время существенно выросло из-за кросс-валидации (модель обучается 5 раз) и дополнительных преобразований.

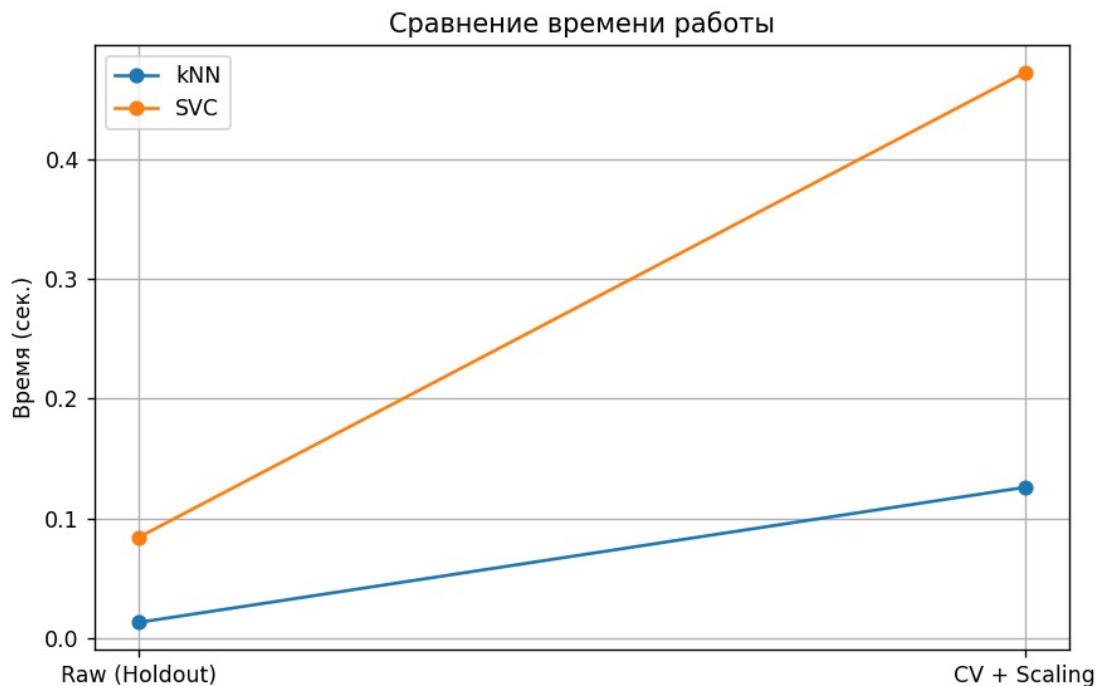
Сводная таблица результатов

Модель	Raw (Holdout) Accuracy	CV + Scaling Accuracy	Raw Time (сек)	CV + Scaling Time (сек)
k-NN	0.8119	0.8320	0.0130	0.1259
SVC	0.8401	0.8395	0.0842	0.4728

Графическая интерпретация



На графике точности видно, что k-NN заметно улучшается после масштабирования, а SVC остаётся примерно на том же уровне.



На графике времени видно, что режим CV + Scaling существенно медленнее, что объясняется многократным обучением моделей (5 фолдов) и этапом стандартизации.

Кластеризация

- Распределение по кластерам:
 - Кластер 0: 383 объектов (24.0%)
 - Кластер 1: 1212 объектов (76.0%)
- Сравнение кластеров с истинными метками (Y):

Cluster	0	1
Y		
0	308	1031
1	75	181

- Оценка качества кластеризации:

Коэффициент силуэта: 0.3104

Интерпретация: средняя структура кластеров

Вывод

1. На сырых данных по метрике accuracy лучше показал себя SVC (0.8401), однако матрица ошибок выявила, что модель все объекты относит к классу 0, то есть качество распознавания класса 1 неудовлетворительное. Следовательно, одной accuracy недостаточно для корректной оценки качества при возможном дисбалансе классов; для более честного анализа следует дополнительно использовать, например, precision/recall/F1 по классу 1.
2. Предобработка в виде StandardScaler дала ожидаемый эффект для k-NN: точность выросла с 0.8119 до 0.8320, что подтверждает чувствительность метода ближайших соседей к масштабу признаков.
3. Для SVC после масштабирования и кросс-валидации точность осталась практически неизменной (0.8401 → 0.8395), что говорит о стабильности результата по accuracy, но не гарантирует хорошее качество по миноритарному классу.
4. Использование кросс-валидации повышает надёжность оценки (усреднение по разным разбиениям), но увеличивает затраты времени: особенно заметно для SVC (до 0.4728 сек).
5. В рамках проведённого эксперимента наиболее корректным выводом является: масштабирование улучшает k-NN, а для SVC требуется более внимательный подбор параметров/метрик, так как модель может “сваливаться” в предсказание одного класса.